

최신 자바스크립트 스터디 가이드

1. 개요 및 웹 아키텍처

1.1 웹 아키텍처

- **웹 애플리케이션(웹사이트):** 웹 서버의 프로그램을 네트워크를 통해 웹 브라우저에서 실행합니다.
- **데스크탑 앱/모바일 앱:** 컴퓨터에 직접 설치되어 실행되는 프로그램입니다.
- **웹 애플리케이션의 처리 구조:**
 1. 웹 브라우저가 웹 서버에 페이지 요청
 1. 웹 서버가 요청을 받아 웹 애플리케이션 서버(WAS)에 로직 및 DB 연동 요청
 1. WAS가 필요시 데이터베이스와 연동 수행
 1. WAS가 처리 결과를 웹 서버에 반환
 1. 웹 서버가 결과를 웹 브라우저에 응답
- **웹 애플리케이션 구성요소:**
 - **웹 브라우저:** 클라이언트, 사용자의 작업 창 (IE, Chrome, Safari, FireFox)
 - **웹 서버:** 클라이언트 요청 웹 페이지 서비스 (Apache, IIS, Tomcat)
 - **웹 컨테이너:** JSP, Servlet 해독 (Tomcat)
 - **웹 애플리케이션 서버(WAS):** 로직 처리, DB 연동 (Tomcat, Jetty, Weblogic, JBoss)
 - **데이터베이스:** 데이터 저장소 (Oracle, Sybase, Informix, DB2, Mssql, MySQL)

1.2 웹 표준 기술 개요

- **웹페이지 구성:**
- **클라이언트 사이드 (UI):** HTML (문서 구조), CSS (태그 표현/반응형), JavaScript (태그 동작 처리/Ajax 구현)
- **서버사이드 (로직 처리, DB 연동):** JSP (컨트롤러: 서블릿, 뷰: JSP, 모델: 자바클래스/자바빈), CGI, ASP, PHP
- **웹페이지 실행:** 웹 브라우저를 사용하여 웹 서버가 제공하는 웹 페이지를 실행합니다.

2. 개발 환경 설정

2.1 서버사이드(JSP) 확장을 고려한 세팅

- **필수 설치:** 웹 브라우저, Java (JDK), Tomcat, Eclipse (Eclipse IDE for JAVA EE Developers)
- **크롬 개발자 도구:** Ctrl+Shift+i로 표시/해제 가능

2.2 클라우드 기반 에디터 (무설치)

- **w3schools.com:** [Try it Yourself] 버튼 클릭 후 코드 수정 및 실행
- **jsbin.com:** [HTML], [Output] 버튼 외 필요시 [CSS], [JavaScript], [Console] 버튼 클릭, 코드 자동 실행
- **Google Colab:** %%html 매직 명령으로 HTML 실행
- **GitHub 레포지토리 웹 서비스:**
 1. 새 레포지토리 생성
 1. index.html 업로드 또는 생성
 1. [Settings] 클릭, [Pages] 항목에서 [Branch]를 main으로 선택
 1. 웹 페이지 배포 주소 확인

2.3 PC에 프로그램 설치

- **JDK (Java Development Kit) 17 LTS:**
- Java 프로젝트, JSP 웹 프로젝트, Android 프로젝트, 이클립스 사용에 필수
- Oracle No-Fee Terms and Conditions (NFTC) 라이선스로 상업적 용도로도 무료 사용 가능
- 다운로드 및 기본값으로 설치
- **환경변수 설정:** PATH (C:\WProgram Files\Java\jdk-17\bin), JAVA_HOME (C:\WProgram Files\Java\jdk-17)
- **Tomcat (웹 컨테이너, 웹 서버, WAS 기능 제공):**
- 다운로드 후 압축 해제
- **환경변수 설정:** CATALINA_HOME (C:\Wapache-tomcat-10.1.5)
- **서버 실행/종료:** bin 폴더 내 startup.bat/shutdown.bat 실행
- **문제 해결:** JAVA_HOME 환경변수 오류 또는 포트 충돌 확인 (netstat -nao)
- **테스트:** 웹 브라우저에서 http://127.0.0.1:8080 접속

- **Eclipse (통합 개발 환경 IDE):**
- Eclipse IDE for Java EE Developers 버전 사용 권장 (다목적 사용 가능)
- 다운로드 받은 zip 파일 압축 해제
- 실행: eclipse.exe 파일 실행
- 워크스페이스 지정: 프로젝트 관리 폴더 지정 (공통 환경 설정 저장)
- 작업 환경 설정: Window - Preferences 메뉴
- 한글 인코딩: General - Workspace - Text file encoding을 UTF-8로 설정
- 글꼴 크기: General - Appearance - Colors and Fonts - Basic - Text Font에서 11~14 사이 지정
- **웹 애플리케이션 작성 (동적 웹 프로젝트):**
- 서버 설정: 이클립스에서 Tomcat 제어 (File - New - Server - Apache - Tomcat 10.1 Server)
- 동적 웹 프로젝트 작성: File - New - Dynamic Web Project (웹 사이트 1개당 프로젝트 1개)
- 프로젝트 서버 등록: Servers 뷰에서 [Add and Remove...] 메뉴로 프로젝트 추가
- 서버 실행/중단: Servers 뷰에서 아이콘 클릭
- 인코딩 변경: Web - CSS, Web - HTML, Web - JSP에서 Encoding을 ISO 10646/Unicode (UTF-8)로 지정
- **웹 프로젝트 파일 위치:**
- 로직 소스 파일 (.java, .class): [프로젝트]-[src]-[main]-[java]
- 뷰 파일 (.html, .jsp, .css, .js), 기타 파일 (이미지, 동영상): [프로젝트]-[src]-[main]-[webapp]
- 환경설정 파일 (web.xml): [프로젝트]-[src]-[main]-[webapp]-[WEB-INF]
- 라이브러리: [프로젝트]-[Java Resources]-[Libraries] (시스템 제공), [프로젝트]-[src]-[main]-[webapp]-[WEB-INF]-[lib] (추가 라이브러리)
- **웹 페이지 템플릿 설정:** Window - Preferences - Web - HTML Files - Editor - Templates - New HTML file (5)에서 뷰포트 메타 태그 추가
- **HTML 웹페이지 작성 및 실행:** [webapp] 폴더에 index.html 생성 후 Run As - Run on Server
- **웹사이트 배포 (WAR):** 실제 환경 서비스 목적
- 프로젝트에서 마우스 우클릭 Export - WAR file
- 대상 경로 (C:\Wapache-tomcat-10.1.5\webapps) 지정 후 Overwrite exist file 체크
- Tomcat 서버 기동 후 웹 브라우저에서 http://127.0.0.1:8080/studyhtml5 접속
- **Eclipse 가상 환경 vs 실제 서비스 환경:**
- 가상 환경: 이클립스 내 웹 서비스 환경 제공 (Servers 항목)
- 실제 서비스 환경: 개발자 PC의 Tomcat 홈 폴더, WAR 파일 배포

3. HTML

3.1 HTML, CSS, JS 개요

- 분리 이유: 유지보수 용이성 (내용과 디자인 분리)
- 웹 화면 작성 순서: HTML (내용) -> CSS (디자인) -> JS (동작) -> 서버 (로직, DB)
- 각 기술의 역할:
- HTML: 문서 내용과 구조 기술 (HTML5 태그로 JS 기능 일부 대체)
- CSS: HTML 태그 표현/디자인 (CSS3로 이미지 대체 기능 추가)
- JS: HTML 문서 내 각종 처리 (이벤트, Ajax, 플래시 대체 - canvas + JS)

3.2 주요 HTML 태그

- <HEAD>: 문서 정보 기술
- <BODY>: 문서 내용 기술
- <meta> 태그: 메타데이터 정의 (페이지 설명, 키워드, 인코딩, 뷰포트)
- charset="UTF-8": 인코딩 설정 (국제 표준 권장)
- viewport 설정: 작은 화면에서 콘텐츠 자동 축소 방지 (width=device-width, initial-scale=1.0 등)
- <script> 태그: JavaScript 코드 정의
- <script>와 </script> 사이에 코드 기술
- src 속성으로 외부 .js 파일 가져오기 (<script src="main.js"></script>)
- 실무 권장: </body> 태그 바로 위에 작성하여 사이트 성능 고려
- <style> 태그: 웹 페이지 스타일 정보 정의

- <style>와 </style> 사이에 CSS 코드 기술
- <link rel="stylesheet" href="main.css">로 외부 .css 파일 가져오기
- <body> 태그 내 주요 태그:
- <p>: 문단 구분
-
: 줄 바꿈
- <i>, : 글꼴 속성 (기울임, 진하게 - CSS에서 지정 권장)
- : 이미지 삽입 (src 속성)
- 리스트 태그:
- : 순서 없는 리스트 (네비게이션 바/메뉴 작성 시 사용)
- <a>: 하이퍼링크 (다른 페이지 또는 같은 페이지 내 특정 위치 이동 - href 속성, id 속성 값 활용)
- 테이블 관련 태그: <table> (표), <tr> (행), <td> (셀) - PC 브라우저 전용 웹페이지에만 사용 권장
- <div>: HTML 문서에서 영역 정의 (id 속성으로 구역 구분)
- HTML5 문서구조 태그 (시맨틱 태그): 문서의 내용과 구조를 명확히 기술
- <header>: 사이트 소개/네비게이션
- <nav>: 네비게이션(메뉴) 항목
- <article>: 문서의 내용(글)
- <section>: 문서의 내용
- <aside>: 본문과 독립적인 사이드바 콘텐츠
- <footer>: 제작자, 주소, 저작권 정보
- <figure> / <figcaption>: 설명글을 붙일 대상 및 설명글
- 웹 폼 태그: 사용자에게 입력받는 화면 구성
- 모든 폼 태그는 <form>과 </form> 사이에 위치 (Ajax 사용 시 생략 가능)
- <label> 태그를 input, select에 붙여 사용 (for 속성)
- <fieldset> / <legend>: 체크박스나 라디오 버튼 그룹화
- <input> type: text, password, checkbox, radio, file, submit, reset, button, email, url, tel, number, range, date, time, week, search, color 등
- <select> / <option>: 여러 항목 중 1개 선택 (콤보상자)
- <textarea>: 여러 줄 입력 텍스트상자
- <button>: 버튼 생성 (이벤트 제어)
- 미디어 및 그래픽 관련 태그:
- 미디어 태그:
- <audio>: 오디오 파일 재생 (controls 속성으로 제어기 표시)
- <video>: 비디오 재생 (width, height, controls 속성)
- <embed>: 플러그인 콘텐츠 표시 (공공기관에서 <iframe> 대신 YouTube 동영상 연결 시 권장)
- <object>: 내장 객체 사용 콘텐츠 표시
- <iframe>: 유튜브 동영상 연결 (보안 이슈로 embed 선호될 수 있음)
- 그래픽 태그:
- <canvas>: 게임 제작 시 사용 (이벤트 반응 그래픽 영역)
- <svg>: 지도 표시 등에 사용

4. CSS

4.1 CSS 기본 개요

- 캐스캐이딩 스타일시트(Cascading Style Sheets): HTML 태그를 화면상에 어떻게 표현할지 설정
- 사용 이유:
- 다양한 화면 크기에 최적의 화면 제공
- 디자인을 별도 파일로 작성하여 체계적인 개발 (내용과 디자인 분리 -> 유지보수 용이)
- 작성 위치:
- 방법1 (비권장): 태그의 style 속성에 직접 기술 (<p style="color:blue;">) - 재사용 불가
- 방법2 (비권장): HTML 문서 내 <style> 태그 안에 기술 - 같은 페이지 내 재사용 가능하나 다른 페이지에 재사용 불가
- 방법3 (권장): 별도의 .css 파일로 작성 (<link rel="stylesheet" href="style.css"/>) - 여러 웹 페이지에서 재사용 가능 (1개 파일로 작성 권장)
- 글꼴 치수: rem (상속 관계에서 상대적 크기, 1rem = 16px)
- CSS 작성 방법:

- **규칙 집합 (rule set):** 선택터와 선언 블록 ({ })으로 구성
- **선택터:** 태그명, id 속성 값, class 속성 값 등
- **선언 블록:** 속성(property)과 값(value)의 쌍 (font-size: 16px; color: blue;)
- 속성과 값은 콜론(:)으로 구분, 선언 사이는 세미콜론(;)으로 구분
- 여러 선택터에 같은 선언 적용 시 콤마(,)로 나열 (div, p {font-size: 16px;})
- 주석: /* 주석 내용 */

4.2 CSS 선택터

- **태그명:** 모든 해당 태그에 적용 (div {color: red;})
- **class 속성 값:** .class속성값 (.a {color: red;})
- **id 속성 값:** #id속성값 (#id3 {color: red;})
- **의사 엘리먼트 (Pseudo-element):** 선택터: hover (마우스 포인터가 위치할 때 적용)
- **조건:** 태그명[조건] (대괄호 사용, 예: p[id] {color: blue;})

4.3 구역 배치

- **float 속성:** 태그를 유동적으로 배치 (left, right)
- **clear 속성:** float 속성 해제 (left, right, both 권장)
- **반응형 웹 구현 필수 요소:**
- float: left; : 화면 크기에 따른 구역 배치
- @media screen 미디어 쿼리: 각 화면 크기별 최적 모양 지정
- %, em, rem: 유동 치수 (기준 화면 사이 간극 처리)

4.4 반응형 웹

- **미디어 쿼리:** 화면 크기를 인식하여 디자인을 적용할 화면의 기준 크기 지정
- **목적:** 최적의 형태로 표현
- **작성 방법:** @media screen and (조건) { /* 스타일 */ }조건: min-width (~이상), max-width (~이하)
- 예: @media screen and (min-width: 320px) { }
- **작성 위치 (권장):** 기존 .css 파일에 함께 기술하여 로딩 속도 고려
- **미디어 쿼리 레벨 4:** 스크립팅, 포인터, 호버, 광도 등을 표현
- scripting: none (기본값), enabled (JS 실행 후)
- pointer: none, coarse (손가락 터치), fine (마우스/펜 터치)
- hover: none, on-demand, hover
- light-level: normal, dim, washed (주변광)
- **유동형 레이아웃:** %, em, rem 등 유동 치수를 사용하여 기준 크기 사이의 간극 처리

4.5 부트스트랩 (Bootstrap)

- HTML, CSS, JavaScript를 사용하여 반응형/모바일 우선 웹사이트를 쉽게 작성할 수 있는 무료 프레임워크 (현재 최신 버전 Bootstrap 5)
- **사용법:**
- 1. <!DOCTYPE html> 및 뷰포트 메타 태그(width=device-width,initial-scale=1.0) 사용
- 1. Bootstrap CDN (jsdelivr.net)을 HTML 페이지에 추가
- 1. div 태그에 class="container" 또는 class="container-fluid"를 사용하여 콘텐츠 담는 컨테이너 작성
- container-fluid: 항상 너비 100%
- container: 화면 너비에 따라 max-width가 다름

5. JavaScript

5.1 JavaScript 개요

- 웹 페이지의 처리 능력 향상 목적 스크립트 언어
- 사용 분야: 웹사이트, 전자기기 제어, 데이터 분석, 인공지능 프로그래밍
- **사용 방법:**
- **HTML 페이지 내:** <script>와 </script> 사이에 코드 기술
- **별도 .js 파일:** <script src="경로/파일명.js"></script>**권장:** <body> 태그의 마지막 위치에 <script> 태그 작성하여 사이트 성능 향상 (가급적 1개 파일로 작성)
- **문장 구조:** 유니코드 기반, 대소문자 구분, 문장 끝은 세미콜론(;), 주석은 // 또는 /**/ 사용
- **리터럴:** 데이터의 값 (숫자, 문자열, 함수, 객체, 배열 등)
- **변수 선언:** var 키워드 사용 (var 변수명 = 리터럴;)
- **식별자:** 변수명, 함수명, 클래스명 등 (첫 글자는 영문자, _, \$; 이후 숫자 사용 가능)

5.2 데이터 타입

- 기본 데이터 타입 (값 저장): 숫자, 문자, 부울, undefined
- 객체 (참조, 레퍼런스) 타입 (객체 주소 저장): 객체, 배열, 함수
- undefined: 변수 타입이 정의되지 않음 (값이 할당되지 않음)
- null: 객체 레퍼런스 변수가 가리키는 객체가 없음 (객체 초기화에 사용)
- empty: 문자열 변수 등의 값이 비어있음 (공백 값)
- 자바스크립트 객체 {}: var 객체변수명 = {키: 값, 메소드: function(){}};
- 배열 []: var 배열변수명 = [원소1, 원소2];
- 함수 function(): var 함수변수명 = function(인수리스트){처리코드};
- 변수: 프로그램에 필요한 값을 일시적으로 메모리에 저장 (파일/DB는 디스크에 영구 저장)
- 객체 생성 방법:
 - Object() 생성자 함수 사용 (비권장)
 - 객체 리터럴 사용 (권장): var obj1 = {};
 - 생성자 함수 사용 (객체 확장/상속 시): var obj1 = new 생성자함수();
- 객체 사용 (접근):
 - 프로퍼티: 객체명.프로퍼티명 또는 객체명['프로퍼티명'] (값 설정/갯팅/새 프로퍼티 생성)
 - 메소드: 객체명.메소드명()
 - 객체 프로퍼티 반복: for(var 변수 in 객체)
 - 객체 프로퍼티 삭제: delete 객체.프로퍼티명
 - 객체 비교: == 연산자는 객체의 주소를 비교 (값이 같아도 다른 객체면 false)

5.3 제어문

- 프로그램 실행 순서 제어: 조건문, 반복문, 기타 제어문
- 조건문:
 - if문: 결과가 2개로 분기 시 사용 (기본 if, else if, else)
 - 복잡한 조건식: && (and), || (or)
 - switch문 (비권장): 조건식 결과가 정수만 가능하여 제한적
 - 조건 연산자 (삼항 연산자): (조건) ? 참 : 거짓; (간단한 조건문)
- 반복문: 반복 작업 처리
 - for문: 지정된 횟수만큼 반복
 - for-in문: 객체 내의 프로퍼티 수만큼 반복
 - for-of문: 컬렉션 내의 객체를 반복해서 처리 (컬렉션: 객체 모임)
 - while문: 조건을 만족하는 동안 반복 (반복 횟수를 알 수 없을 때 선호, for보다 느림)
 - do-while문: 일단 1번 실행 후 조건 비교 반복
- 루프 탈출문:
 - break문: 반복문 완전 탈출 (무한루프 중단 시 주로 사용)
 - continue문: 일시적으로 탈출했다가 다시 반복문 수행

5.4 함수

- 특정 로직 처리에 사용, 재사용 가능
- 종류: 시스템 제공 함수 (내장 함수), 사용자 정의 함수
- 함수 원리: 함수명, 인수 개수/타입 (인수 리스트), 실행 결과 타입 (리턴 타입)
- 내장 함수: 시스템이 제공하는 함수 (alert() 등)
- 사용자 정의 함수:
 - 일반 함수: function 문 사용
 - 함수 선언문: function 함수명(매개변수리스트){}
 - 함수 표현식 (권장): var 함수명 = function(매개변수리스트){}; (체계적인 프로그램 작성, 후방 참조 불가)
 - Function() 생성자 함수 사용 (비권장)
- 익명 함수 (이름 없는 함수): 이름을 굳이 줄 필요가 없는 경우 (이벤트 처리, 즉시 실행 함수, 클로저, 함수 표현식)
- 변수에 저장하거나 함수의 인수로 사용
- 콜백 함수: 이벤트에 반응하는 함수 (이벤트 처리 시 자동 실행)
- 즉시 실행 함수 (Self-Invoking Functions): 함수 정의와 동시에 실행 ((function() { //내용 })()); - 초기화 등에 사용 (최초 1번만 실행)
- 클로저: 함수의 캡슐화, 런타임 실행 시 인자 값 줘서 수행할 때 사용 (함수 안에 리턴값으로 정의된 익명 함수)
- 함수 호출법: 함수명(인수리스트);

5.5 클로저

- 즉시 실행 내부 함수로 함수의 캡슐화, 런타임 실행 시 인자 값을 넘겨서 자동 동작하는 작업

에 사용

- **개요:** 함수 안에 리턴 값으로 정의되는 익명 함수
- **전역 변수와 지역 변수 스코프 문제:**
- **전역 변수:** 웹 페이지 내에서 긴 스코프를 가짐 (보안 문제 발생 가능)
- **지역 변수:** 해당 함수의 사용이 끝나면 사라짐
- **카운터 딜레마:**
- 카운터 변수를 전역 변수로 선언 시 보안 문제
- 카운터 변수를 지역 변수로 선언 시 함수 호출마다 초기화되어 카운팅 불가
- **클로저를 사용하여 해결:** 즉시 실행 함수와 내부 함수 결합으로 counter 변수를 보호하고 값 누적 가능

5.6 객체

- 실무 데이터 처리를 위해 사용 (데이터를 객체로 저장하여 원하는 위치까지 전달 가능)
- **개요:** 실무 데이터 1건, 속성 (프로퍼티/멤버 필드), 메소드
- **객체 생성:** var 객체변수명 = {속성: 값, 메소드: function(){} }; (객체 리터럴 사용)
- **속성/메소드 사용:** 객체명.속성명, 객체명.메소드명()
- **속성 반복:** for (var 속성저장변수 in 객체명){}
- **생성자 함수:** 객체 생성 틀 (객체 확장/상속에 사용)
- 프로퍼티 및 메소드 정의 시 this 키워드 사용 (this.name = name:)
- 프로토타입 밖에서 메소드 정의: 프로토타입명.prototype.함수명 = function(){};

5.7 JavaScript 프로그램 작성 시 권고사항 (JS Best Practices)

1. 전역 변수 사용 자제
1. 지역 변수 선언 후 사용: var a = 0;
1. 먼저 선언하고 사용 (Declarations on Top)
1. 변수 초기화: var a = 0; , var b = "";
1. 숫자, 문자, 불린 값은 객체 형태로 쓰지 말 것 (성능 저하)
1. new Object(); 문장 쓰지 말 것: {} 사용
1. 자동 형변환에 주의

5.8 내장 객체

- 기본 객체: Number, String, Array, Date
- 문서 객체 모델 (DOM): 웹 페이지의 HTML 요소를 객체로 표현하여 JavaScript로 조작할 수 있게 해주는 API
- Window, Document, Screen, History, Location, Navigator, Style 등
- **태그(엘리먼트) 접근 메소드:**
- document.getElementById("id속성값") (주어진 ID를 가진 태그에 접근)
- document.getElementsByTagName("태그명") (주어진 태그명을 가진 모든 태그 객체를 배열로 리턴)
- **태그 내용 접근:** 태그객체.innerHTML 속성 사용
- 내용 얻기: var v1 = document.getElementById("id속성값").innerHTML;
- 내용 넣기: document.getElementById("id속성값").innerHTML = "내용";

5.9 예외 처리

- 프로그램을 안정적으로 구현하기 위해 사용
- **try-catch-finally:**
- try: 예외를 유발할 수 있는 코드
- catch(exception): 예외 발생 시 처리할 코드
- finally: 예외 발생 여부와 상관없이 반드시 실행할 코드 (리소스 해제)

6. Ajax와 jQuery

6.1 Ajax의 개요 및 구성요소

- **개요:** "Asynchronous JavaScript+XML"의 약어로, 웹 브라우저 자체의 기능(Javascript, DOM, 스타일시트)을 사용하여 비동기적으로 서버와 통신하는 기술
- 구글 서제스트(검색어 자동 완성)나 구글 맵 등에서 사용
- **구성요소:**
- XHTML(HTML5)과 CSS를 사용한 표준 기술 기반 웹 페이지
- DOM을 사용한 동적인 화면 표시와 상호작용 (DynamicHTML)
- XML과 XSLT 등을 사용한 데이터의 변경과 조작 (XML은 태그 정의 가능, HTML보다 문법 규칙 강함)

- XMLHttpRequest를 사용한 비동기적인 데이터 전송 (HTTP 통신)
- 위 구성요소들을 결합해서 사용하는 JavaScript
- **비동기적 통신 구조:**
 1. 웹 브라우저에서 사용자 동작으로 서버에 요청 송신 (XMLHttpRequest 객체 사용)
 1. 웹 브라우저는 응답을 기다리지 않고 다른 작업 처리 가능
 1. 서버는 요청 처리, DB 연동
 1. 서버가 웹 서버에 결과 반환
 1. 웹 서버는 필요한 데이터만 웹 브라우저에 응답 (페이지 전체가 아닌 일부만 업데이트)
- **XMLHttpRequest 객체:**
 - Ajax에서 서버와 웹 브라우저 간 통신 담당하는 가장 중요한 객체
 - `const xhttp = new XMLHttpRequest();`로 생성
 - **메소드:** `abort()`, `getAllResponseHeaders()`, `getResponseHeader()`, `open()`, `send()`, `setRequestHeader()`
`open(method, url, async, userName, password):` 요청 설정 (method, url 필수)
 - `send(content):` 요청 서버로 전송 (GET 방식은 null, POST 방식은 DOM 객체, 문자열 등)
 - **프로퍼티:** `onreadystatechange`, `readyState`, `responseText`, `responseXML`, `responseBody`, `status`, `statusText`
`readyState:` 요청 객체의 상태 (0: uninitialized ~ 4: loaded)
 - `status:` 서버로부터 응답받는 HTTP 상태 코드 (200: OK, 404: Not Found, 500: Server Error)
 - `readyState == 4 && status == 200`일 때 정상 응답으로 간주하여 처리

6.2 jQuery

- JavaScript 라이브러리: JavaScript를 쉽고 간단하게 사용하는 방법 제공
- HTML/DOM 작업, CSS 작업, HTML 이벤트 처리, 효과/애니메이션, Ajax, 유틸리티 등
- **사용 방법:**
 1. jQuery 라이브러리 다운로드 후 프로젝트에 추가 (`<script src="../../js/jquery-3.1.0.js"></script>`)
 1. jQuery CDN 경로를 HTML 페이지에 추가 (`<script src="https://code.jquery.com/jquery-3.1.0.js"></script>`)
- **기본 사용법:**
 - 문법: `$(작업대상태그).어떤동작(function(){//제어할 작업});`
 - `$(selector):` 제이쿼리에서 태그/객체 접근에 사용
 - `selector:` 태그명 (`$("#p")`), class 속성값 (`$(".round")`), id 속성값 (`$("#p1")`), 객체명 (`$(document)`)
 - `action():` 작업 대상에서 발생한 동작 (이벤트) 처리 (`hide()`, `show()`, `click()`, `ready()`)
 - **코드 작성 방법:** `$(document).ready(function(){//코드});` 안에 작성 (페이지 로딩 완료 후 자동 실행)
- **HTML 엘리먼트 내용 접근 (get/set):**
 - `text():` 엘리먼트 내용을 텍스트 형태로 지정하거나 얻어냄
 - `html():` 엘리먼트 내용을 HTML 태그 포함하여 지정하거나 얻어냄
 - `val():` 폼 필드의 값을 지정하거나 얻어냄
- **jQuery 이벤트 연결 방법:**
 - `이벤트발생태그.이벤트종류(function()){}`;
 - `이벤트발생태그.on(이벤트종류, function()){}`; (`on()`은 아직 생성되지 않은 자식 엘리먼트에도 이벤트 연결 가능)
 - `이벤트발생태그.on({이벤트종류: function(){}, ...});` (다양한 이벤트 연결)
 - `off():` 이벤트 제거
- **jQuery에서 서버사이드 요청 (Ajax):**
 - 서버로부터 TEXT, HTML, XML, JSON 형태의 파일을 요청하고 응답받음
 - `.load():` 서버로 데이터 요청 후 HTML 엘리먼트에 결과 로드 (로그인 폼, 회원가입 폼 등)
 - `$.get():` HTTP GET 방식 요청
 - `$.post():` HTTP POST 방식 요청
 - `$.getJSON():` HTTP GET 방식으로 JSON 데이터 요청
 - `$.ajax():` 비동기 Ajax 요청 수행 (GET, POST 방식 지정 가능) (로그인 처리, 회원가입 처리 등 DB 연동 후 결과 반환 시 주로 사용)

7. 그 밖의 주요 라이브러리 및 프레임워크

7.1 라이브러리

- **Node.js:**
- **특징:** 푸시 서버 개발에 주로 사용, CPU를 많이 사용하지 않는 가벼운 작업 위주 개발 권장
- **단점:** 콜백 중점으로 유지보수 어려움, V8 엔진 가비지 컬렉션으로 CPU 사용을 정점 시 서버 일시 멈춤 가능
- **장점:** 서버사이드 진입 장벽 낮음, socket.io를 사용한 양방향 통신 용이 (푸시 구조 구현에 좋음)
- **단점:** 단일 스레드 모델로 작업 시간 길어지면 시스템 성능 저하, DB 트랜잭션 많은 앱은 WAS보다 성능 저하
- **설치:** nodejs.org에서 권장 버전 다운로드 설치
- **이클립스 세팅:** nodeclipse 플러그인 설치 후 Node 프로젝트 작성

7.2 프레임워크

- **AngularJS (AngularJS 1.X):** JavaScript 기반 오픈 소스 프론트엔드 웹 애플리케이션 프레임 워크 (클라이언트 사이드 MVC, MVVM 구조 제공)
- **React와 Flux (페이스북 개발):**
- **React:** 프론트엔드 라이브러리
- **Flux:** 애플리케이션 아키텍처
- 고도화된 애플리케이션 구조화를 jQuery보다 쉽게 가능
- **Vue.js:** 사용자 인터페이스를 만들기 위한 진보적인 프레임워크 (점진적으로 채택 가능, 뷰 레이어에 초점, 다른 라이브러리/기존 프로젝트와 통합 용이)

8. AI를 사용한 앱 작성 (Google Gemini/AI Studio)

8.1 Google Gemini를 사용한 웹 사이트 구축

1. Gemini에서 [Canvas] 선택 후 웹사이트 내용 입력
1. [만들기] - [웹사이트]
1. [공유] - [콘텐츠 복사]
1. 코드를 index.html로 저장
1. GitHub 레포지토리 생성 후 index.html 업로드
1. GitHub 레포지토리를 웹 서비스 호스팅으로 변환 (Pages 설정)

8.2 Google AI Studio를 사용한 앱 작성

1. **API 키 받기:** AI Studio 홈에서 [Get API key] 클릭 후 [+API 키 만들기]
1. **챗봇 다운로드 후 외부 서비스:** 챗봇에서 [< >Get Code] 클릭 후 [Download] 또는 [Copy]
1. **[Build]의 앱 다운로드:** [Download app] 클릭
1. **다운로드 받은 앱 로컬 실행:**
 - Node.js 설치 (<https://nodejs.org/ko/download>)
 - 명령 프롬프트에서 앱 경로로 이동 후 npm install
 - .env.local 파일과 package.json 파일 수정 (IP 주소 등)
 - npm run dev 입력 후 웹 브라우저에서 http://localhost:5173/ 또는 http://자신의ip:5173/ 접속

퀴즈 (10개 단답형 질문)

지시: 다음 각 질문에 대해 2~3문장으로 간략하게 답변하시오.

1. 웹 애플리케이션이 작동하는 기본적인 처리 구조를 설명하고, 이 과정에서 웹 브라우저의 역할을 간략하게 언급하시오.
1. XMLHttpRequest 객체가 Ajax에서 핵심적인 역할을 하는 이유는 무엇이며, 이 객체를 사용하여 요청을 보낼 때 open() 메소드와 send() 메소드의 기본적인 용도를 설명하시오.
1. JavaScript에서 undefined, null, empty 값의 차이점을 각각 간략하게 설명하시오.
1. HTML, CSS, JavaScript를 분리하여 작성하는 것이 웹 개발에서 권장되는 이유는 무엇인지 설명하시오.
1. jQuery 라이브러리를 사용하는 주된 목적 두 가지와 JavaScript 기본 문법과 jQuery 문법의 차이점을 하나의 예시를 들어 설명하시오.
1. JavaScript의 클로저(Closure)가 사용되는 주요 목적 두 가지를 설명하고, 이것이 전역 변수와 지역 변수의 스코프 문제를 어떻게 해결하는지 간략하게 언급하시오.
1. HTML5 문서구조 태그(시맨틱 태그)를 사용하는 이점은 무엇이며, 그 예시로 <header>와

<footer> 태그의 역할을 설명하시오.

1. 반응형 웹 디자인에서 미디어 쿼리(@media screen)가 담당하는 역할은 무엇이며, min-width와 max-width 조건이 어떻게 활용될 수 있는지 설명하시오.
1. Node.js가 서버사이드 개발에서 가지는 장점과 단점을 각각 한 가지씩 설명하시오.
1. 이클립스에서 동적 웹 프로젝트를 서버에 배포할 때 WAR 파일의 목적은 무엇이며, WAR 파일을 생성하는 주된 이유를 설명하시오.

퀴즈 정답

1. 웹 애플리케이션은 웹 브라우저가 웹 서버에 페이지를 요청하면, 웹 서버는 WAS를 통해 로직 및 DB 연동을 처리하고 그 결과를 다시 브라우저에 응답합니다. 웹 브라우저는 이 과정에서 클라이언트로서 사용자의 요청을 보내고 서버로부터 받은 결과를 화면에 표시하는 역할을 합니다.
1. XMLHttpRequest 객체는 Ajax에서 서버와 웹 브라우저 간의 비동기 통신을 담당하는 핵심 객체입니다. open() 메소드는 요청 방식(GET/POST), URL, 비동기 여부 등을 설정하며, send() 메소드는 설정된 요청을 실제로 서버로 전송하는 역할을 합니다.
1. undefined는 변수가 선언되었지만 값이 할당되지 않아 타입이 정의되지 않은 상태를 의미합니다. null은 객체 레퍼런스 변수가 특정 객체를 가리키지 않음을 명시적으로 나타내며, empty는 문자열 변수 등이 비어있는(공백) 상태를 의미합니다.
1. HTML, CSS, JavaScript를 분리하여 작성하는 것은 내용과 디자인, 동작을 각각 독립적으로 관리함으로써 웹 사이트의 유지보수성을 크게 향상시킵니다. 이는 코드의 가독성을 높이고 특정 부분을 수정할 때 다른 부분에 미치는 영향을 최소화하는 데 도움을 줍니다.
1. jQuery를 사용하는 주된 목적은 JavaScript 코드를 더 쉽고 간단하게 작성하여 HTML/DOM 조작, 이벤트 처리, Ajax 기능 등을 구현하는 것입니다. 예를 들어, 특정 ID를 가진 태그에 접근할 때 JavaScript 기본 문법은 document.getElementById("myId")인 반면, jQuery 문법은 \$("#myId")와 같이 간결하게 표현할 수 있습니다.
1. JavaScript의 클로저는 주로 함수의 캡슐화를 구현하고 변수의 스코프 문제 및 카운터 딜레마를 해결하는 데 사용됩니다. 즉시 실행 함수와 내부 함수를 결합하여 내부 변수를 외부에서 직접 접근할 수 없게 보호하면서도, 내부 함수를 통해 그 변수의 값을 누적하거나 변경할 수 있게 합니다.
1. HTML5 문서구조 태그(시맨틱 태그)는 문서의 내용과 구조를 명확하게 기술하여 검색 엔진 최적화(SEO) 및 웹 접근성을 향상시킵니다. 예를 들어, <header> 태그는 사이트의 소개나 내비게이션을 표시하고, <footer> 태그는 사이트의 제작자, 주소, 저작권 정보 등을 표시하여 각 영역의 의미를 명확히 전달합니다.
1. 반응형 웹 디자인에서 미디어 쿼리는 화면 크기를 인식하여 특정 기준 크기에 따라 최적의 디자인과 스타일을 적용하는 역할을 합니다. min-width는 특정 너비 이상일 때 스타일을 적용하고, max-width는 특정 너비 이하일 때 스타일을 적용하여 다양한 디바이스 환경에 맞춰 콘텐츠를 재배치할 수 있게 합니다.
1. Node.js의 장점으로는 서버사이드 개발 진입 장벽이 낮고 socket.io를 사용한 양방향 통신 구현이 쉽다는 점이 있습니다. 반면, 단일 스레드 모델로 인해 하나의 작업에 시간이 많이 소요되면 전체 시스템 성능이 저하될 수 있다는 단점이 있습니다.
1. 이클립스에서 동적 웹 프로젝트를 배포할 때 WAR 파일은 실제 서비스 환경에서 웹사이트를 서비스하기 위한 목적으로 사용됩니다. WAR 파일은 작성한 웹사이트 전체를 포함하는 압축 파일 형태로, 이를 통해 웹 애플리케이션을 쉽게 배포하고 관리할 수 있습니다.

에세이 형식 질문

1. 웹 애플리케이션의 클라이언트 사이드 기술 (HTML, CSS, JavaScript)과 서버사이드 기술 (JSP)의 상호작용 및 각각의 핵심적인 역할에 대해 심층적으로 설명하시오. 이들이 협력하여 동적인 웹 페이지를 사용자에게 제공하는 과정을 구체적인 예시와 함께 서술하시오.
1. JavaScript의 객체 개념과 객체 생성 방법(객체 리터럴, 생성자 함수)에 대해 자세히 설명하시오. 특히 클로저(Closure)가 JavaScript의 변수 스코프 문제와 카운터 딜레마를 어떻게 해결하며, 이 해결 방식이 갖는 이점은 무엇인지 구체적인 코드 예시를 들어 논하시오.
1. 반응형 웹 디자인 구현을 위한 CSS의 핵심 요소인 미디어 쿼리, float 속성, 그리고 유동형 레이아웃(% , em, rem)에 대해 각각 설명하고, 이들이 어떻게 상호 보완적으로 작동하여 다양한 화면 크기에 최적화된 웹 페이지를 제공하는지 분석하시오. 또한 Bootstrap과 같은 프레임워크가 반응형 웹 개발을 어떻게 돕는지 언급하시오.
1. Ajax가 기존 웹 페이지 통신 방식과 다른 점은 무엇이며, XMLHttpRequest 객체의 주요 메소드 및 프로퍼티를 활용하여 비동기적 통신을 구현하는 과정을 단계별로 설명하시오. 또한 jQuery 라이브러리가 Ajax 구현을 어떻게 간소화하는지 \$.ajax() 메소드 사용 예시와 함께 비교 설명하시오.
1. 최신 웹 개발 환경 설정 과정에서 JDK, Tomcat, Eclipse를 설치하고 연동하는 절차를 상세히 설명하시오. 특히 각 구성요소가 수행하는 역할과 환경 변수(JAVA_HOME, CATALINA_HOME) 설정의 중요성을 강조하고, 이클립스 내에서 동적 웹 프로젝트를 생성하고 배포하기까지의 전반적인 과정을 서술하시오.

핵심 용어 사전 (Glossary)

- **웹 애플리케이션 (Web Application):** 웹 서버의 프로그램을 네트워크를 통해 웹 브라우저에서 실행하는 프로그램. 웹을 기반으로 동작하는 소프트웨어.
- **웹 서버 (Web Server):** 클라이언트(웹 브라우저)의 요청을 받아 웹 페이지를 서비스(제공)하는 소프트웨어 또는 하드웨어.
- **WAS (Web Application Server):** 웹 서버와 데이터베이스 사이에서 애플리케이션 로직을 처리하고 동적인 콘텐츠를 생성하는 서버.
- **웹 컨테이너 (Web Container):** JSP와 Servlet을 해독하고 실행하는 환경을 제공하는 WAS의 구성요소.
- **HTML (HyperText Markup Language):** 웹 문서의 내용과 구조를 태그를 사용하여 기술하는 마크업 언어.
- **CSS (Cascading Style Sheets):** HTML 태그를 화면상에 어떻게 표현할지(디자인, 레이아웃) 설정하는 스타일 시트 언어.
- **JavaScript (JS):** HTML 문서에서 발생하는 각종 동작을 처리하고, 서버에 요청(Ajax)을 보내는 등의 기능을 구현하는 스크립트 언어.
- **Ajax (Asynchronous JavaScript and XML):** JavaScript를 사용하여 서버와 비동기적으로 데이터를 주고받아 웹 페이지의 일부분만 업데이트하는 기술.
- **jQuery:** JavaScript 코드를 더 쉽고 간단하게 작성할 수 있도록 돕는 JavaScript 라이브러리.
- **DOM (Document Object Model):** 웹 페이지의 HTML 요소를 객체 모델로 표현하여 JavaScript로 조작할 수 있게 해주는 API.
- **XMLHttpRequest:** Ajax에서 서버와 웹 브라우저 간의 비동기 통신을 담당하는 핵심 객체.
- **JDK (Java Development Kit):** Java 기반 프로젝트(Java, JSP, Android) 개발에 필요한 도구와 라이브러리를 포함하는 개발 키트.
- **Tomcat:** Apache 소프트웨어 재단에서 개발한 오픈 소스 웹 서버이자 서블릿 컨테이너(웹 컨테이너). WAS 역할도 수행.

- **Eclipse**: 다양한 프로그래밍 언어를 지원하는 통합 개발 환경(IDE, Integrated Development Environment).
- **WAR (Web Application Archive)**: 웹 애플리케이션을 배포하기 위한 압축 파일 형식. 실제 서비스 환경에 웹사이트를 올릴 때 사용.
- **반응형 웹 (Responsive Web)**: 화면의 크기에 맞춰 콘텐츠와 레이아웃이 자동으로 재배치되어 최적의 화면을 제공하는 웹 디자인 방식.
- **미디어 쿼리 (Media Query)**: CSS에서 화면 크기, 해상도 등 디바이스의 특성에 따라 다른 스타일을 적용할 수 있도록 하는 기능.
- **부트스트랩 (Bootstrap)**: HTML, CSS, JavaScript를 사용하여 반응형 및 모바일 우선 웹사이트를 쉽게 만들 수 있는 무료 프레임워크.
- **리터럴 (Literal)**: 프로그래밍에서 데이터의 값을 직접 표현하는 방식 (예: 숫자 1, 문자열 "hello").
- **변수 (Variable)**: 프로그램에서 필요한 값을 일시적으로 저장하는 메모리 공간의 이름.
- **객체 (Object)**: 실무 데이터를 표현하는 단위로, 속성(프로퍼티)과 동작(메소드)으로 구성됨.
- **클로저 (Closure)**: 함수와 함수가 선언된 어휘적 환경의 조합으로, 함수가 생성될 당시의 환경을 기억하는 함수. 변수 스코프 문제 해결 및 캡슐화에 사용.
- **컨테이너 (Container)**: 웹 페이지의 내용을 담는 영역을 지정하는 데 사용되는 부트스트랩 클래스 (.container, .container-fluid).
- **float**: CSS 속성으로, 요소를 문서의 일반적인 흐름에서 제거하고 좌우로 띄워 배치하는 데 사용.
- **clear**: CSS 속성으로, float 속성이 적용된 요소의 영향을 받지 않도록 하는 데 사용.
- **시맨틱 태그 (Semantic Tag)**: HTML5에서 도입된, 태그 자체가 의미를 가지는 태그 (예: <header>, <footer>, <article>).
- **Node.js**: Chrome V8 JavaScript 엔진으로 빌드된 JavaScript 런타임으로, 확장성 있는 네트워크 애플리케이션(특히 서버) 개발에 사용.
- **AngularJS / React / Vue.js**: JavaScript 기반의 프론트엔드 웹 애플리케이션 프레임워크 또는 라이브러리로, 복잡한 UI 개발을 효율적으로 돕습니다.